

## Exam 1 Problems by Suthep Madarasmi, PhD.

### 1. Use the following requirements to complete various problems in this section

#### Requirements: KU Course Registration System

- |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>1. All users must log in with their KU account through KU Single Sign-On. User roles such as student, university admin, and instructor are provided by this system.</p> <p>2. Department Admins can create new course information: course code, course name, description, credits, and fee.</p> <p>3. Department Admins can set course offerings by first selecting a semester (e.g., 2025-1) and course code (e.g., CPE334). They then enter a sections list with details: section number, room, class time (e.g., “M W F 10:30–12:00”), display notes (e.g., ผู้สอน, หมายเหตุ), and maximum students allowed. For example, if CPE334 is offered in semester 2025-1 with sections 1, 2, 3, and 4, the system will create four course offerings for CPE334.</p> <p>4. University Admins can create new semesters (e.g., 2024-3, 2025-1, 2025-2) and set the currently active semester.</p> <p>5. Students can open a Registration Form for the active semester. Initially, the form has no course offerings selected. The form automatically pulls data from the KU Student Info System using the logged-in username, displaying student ID, name, degree program, email, login username, etc.</p> <p>6. Students can add multiple course offerings to the Registration Form through the Course Offering List View. The total registration fee updates as courses are added.</p> <p>7. The Course Offering List View displays: course ID, course code, course name, credits, fee, semester, section, room, class time, display notes, maximum students, number registered, and seats remaining. Students can filter and select courses to add to the registration form.</p> | <p>1. user ทุกคนต้อง log in ด้วย KU account ผ่าน KU Single Sign-On โดยที่ User roles เช่น student, university admin และ instructor จะมาจากระบบนี้</p> <p>2. Department Admins สามารถสร้างข้อมูล course ใหม่ได้: course code, course name, description, credits และ fee</p> <p>3. Department Admins สามารถกำหนด course offerings ได้โดยเลือก semester (เช่น 2025-1) และ course code (เช่น CPE334) ก่อน จากนั้นกรอก sections list พร้อมรายละเอียด: section number, room, class time (เช่น “M W F 10:30–12:00”), display notes (เช่น ผู้สอน, หมายเหตุ) และจำนวน maximum students allowed ตัวอย่างเช่น ถ้า CPE334 เปิดใน semester 2025-1 พร้อม sections 1, 2, 3 และ 4 ระบบจะสร้าง course offerings สำหรับ CPE334 ทั้งหมด 4 รายการ</p> <p>4. University Admins สามารถสร้าง semester ใหม่ (เช่น 2024-3, 2025-1, 2025-2) และกำหนด semester ที่ เป็น currently active ได้</p> <p>5. Students สามารถเปิด Registration Form สำหรับ active semester ได้ โดยเริ่มแรก form จะยังไม่มี course offerings ถูกเลือก Form จะดึงข้อมูลจาก KU Student Info System โดยใช้ logged-in username เพื่อแสดง ข้อมูล student ID, name, degree program, email, login username ฯลฯ</p> <p>6. Students สามารถเพิ่ม course offerings หลายรายการ ลงใน Registration Form ผ่าน Course Offering List View โดยที่ total registration fee จะอัปเดตตาม courses ที่ถูกเพิ่ม</p> <p>7. Course Offering List View จะแสดง: course ID, course code, course name, credits, fee, semester, section, room, class time, display notes, maximum students, number registered และ seats remaining โดย Students สามารถ filter และเลือก courses เพื่อเพิ่มลงใน registration form ได้</p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

8. The Registration Form begins in Draft status.  
Students can save and later return to edit drafts before final submission.

9. Upon submission, the system checks seat availability for each course:  
– If a section is full → submission fails, and a warning appears for students to correct and resubmit.  
– If all sections have seats → proceed to payment.

10. Payment options include:  
– Credit card via KBank payment gateway.  
– Bank transfer via Bangkok Bank payment gateway.

11. After successful payment, the registration status becomes Done:  
– Forms with Done cannot be edited online.  
– Students receive a PDF copy of their registration by email via KU Email (Google Workspace).

8. Registration Form จะเริ่มต้นในสถานะ Draft โดย Students สามารถ save และกลับมา edit draft ก่อนการ submit ครั้งสุดท้ายได้

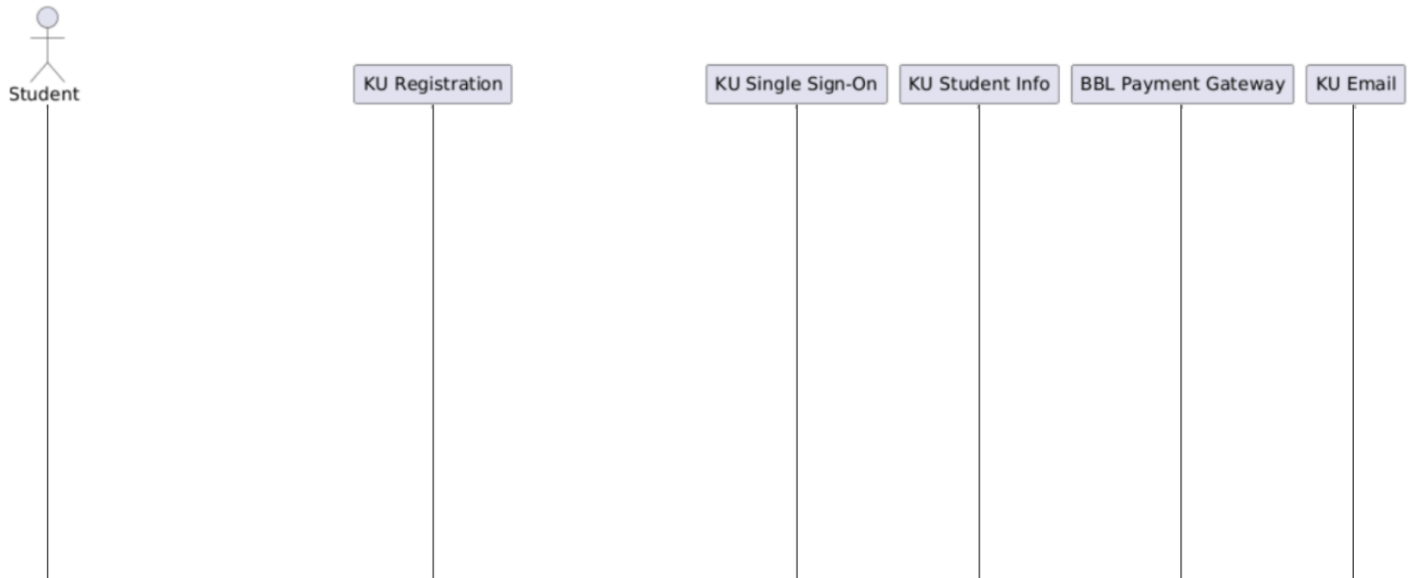
9. เมื่อ submit ระบบจะตรวจสอบ seat availability ของแต่ละ course:  
– ถ้า section เต็ม → submission ล้มเหลว และจะมี warning ปรากฏเพื่อให้ Students แก้ไขและ resubmit  
– ถ้าทุก section ยังมีที่นั่ง → ดำเนินการต่อไปที่ขั้นตอน payment

10. Payment options ได้แก่:  
– Credit card ผ่าน KBank payment gateway  
– Bank transfer ผ่าน Bangkok Bank payment gateway

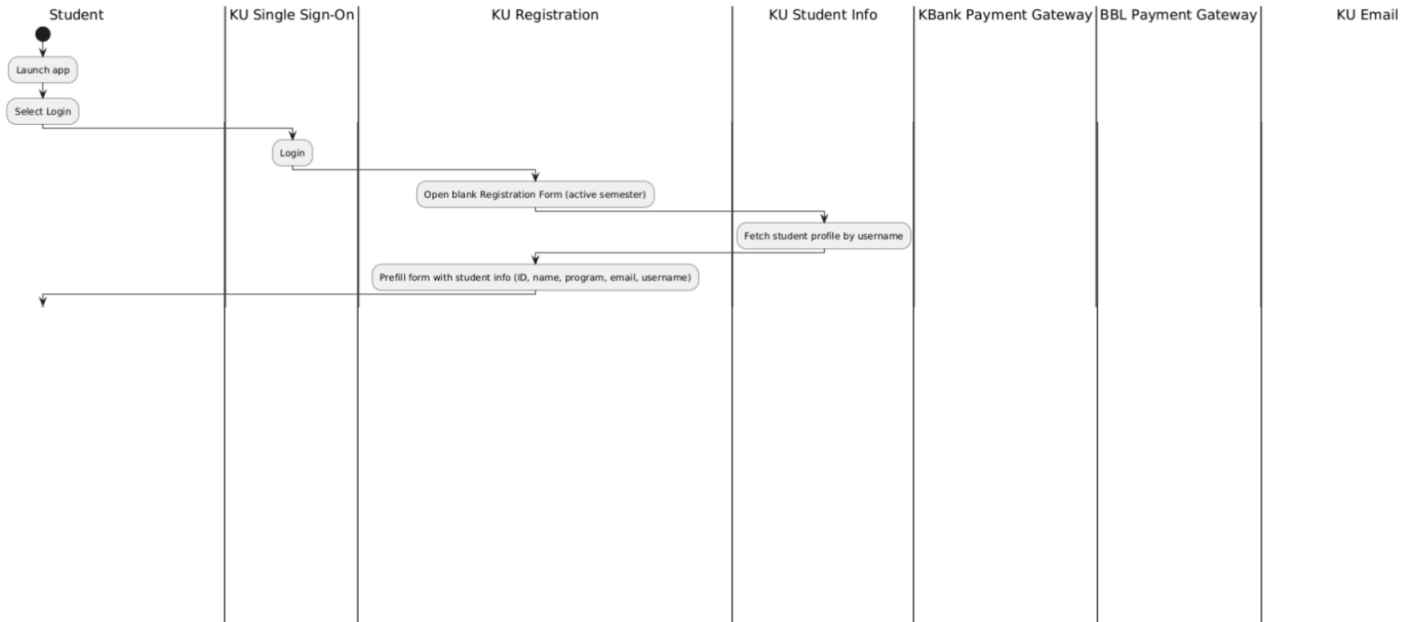
11. หลังจากการ payment สำเร็จ registration status จะกลายเป็น Done:  
– Form ที่เป็น Done จะไม่สามารถ edit ออนไลน์ได้  
– Students จะได้รับ PDF copy ของ registration ทาง email ผ่าน KU Email (Google Workspace)

**1.1 10 points. Complete this sequence diagram with swimlanes for the following end-to-end scenario:**

1. A student logs in.
2. The system detects that the student already has a Draft Registration Form for the current semester containing two course offerings, so the student is presented with that form to continue.
3. The student adds one more course offering.
4. The student submits the form, but one of the three course offerings is full so submission fails.
5. The student is informed.
6. The student changes that course offering to a different section and submits again.
7. This time the submission is successful.
8. The student pays by bank transfer.
9. The registration is completed successfully.



1.2 10 points. Complete the UML Activity Diagram with partitions (swimlanes) shown below. It models the main registration workflow in which a student logs in for the first time this semester, receives a blank registration form containing their information, selects courses, pays successfully, and receives confirmation. Be sure to cover all decision branches (e.g., payment method) and loops (e.g., resubmit until confirmation is received) using simple flowchart notation for decision nodes. The workflow represents a single session and ends with the student successfully registered.



1.3 10 points. Design a generic UI for the Student Registration Form View. Assume the semester is fixed to the current semester and student information is loaded from the logged-in user. Show required UI fields using leading “\*”, editable fields as textboxes, non-editable fields as striped textboxes as shown in the example below.

[Save] [Clear] [Close]

Invoice No.

\*Invoice Date:

[Date]

\*Customer Code:

[LoV]

Customer Name:

Notes:

|   | *Product Code | Name  |  | *Quantity | Unit Price |  | Price |     |
|---|---------------|-------|--|-----------|------------|--|-------|-----|
| 1 |               | [LoV] |  |           |            |  |       | x ▲ |
| 2 |               |       |  |           |            |  |       | x □ |
| 3 |               |       |  |           |            |  |       | x ▼ |

Total Due:

**1.4 10 points. Draw the Class diagram for the data model used to store the Registration Form and its immediate (first level) links to other classes. Do not include second level links (i.e., anything more than one hop away from Registration Form's Header and Line Item classes). Show attributes, external calls, associations, multiplicities, inheritance, and composition. Do not show the class methods. Class Diagrams will be covered in the Final Exam.**

## 2. Git Workflow Problems

### Project Context

ExamScore is a small Python command-line application used by a university instructor. It loads student names and exam scores from a CSV file, checks that scores are between 0 and 100, calculates basic statistics, and prepares a pass-rate report. Python was chosen because its syntax is easy to read and it supports CSV processing and numerical calculations without distracting from the Git workflow.

The branch main represents the deployed release. Sprint staging branches collect completed work for integration and testing. Feature branches isolate individual changes until the work is ready to merge.

### Files Used in the Problems

| File                 | Purpose                                          |
|----------------------|--------------------------------------------------|
| score_data.py        | Python code that loads and validates exam scores |
| scores.csv           | Sample student names and score data              |
| validation_notes.txt | A new file documenting validation test cases     |

### Diagram Notation and Assumptions

- HEAD identifies the currently checked-out local branch; thus, the commit shown in the working directory.
- A name such as origin/main is a local remote-tracking reference. It records the last GitHub state known to this PC.
- Switching branches moves HEAD but does not move any branch pointer.
- Assume the working directory is clean, all commands succeed, and no merge conflict occurs unless the problem states otherwise.
- No command communicates with GitHub unless fetch, push, or another remote command is shown.
- Use explicit file names when staging. Do not use git add . in these problems.

## 2.1 Predict the New Repository Graph 10 points

Developer 1 is working in a local ExamScore repository. The starting graph is shown below. The three commands are then run in the order shown.

```
A---B [main, origin/main]
  \
   C [sprint_1_staging, origin/sprint_1_staging]
   \
    D---E [HEAD -> feature/load_scores]
```

### Commands

```
> git switch sprint_1_staging
> git merge feature/load_scores
> git switch main
```

**Task:** Draw the final repository graph. Show main, sprint\_1\_staging, feature/load\_scores, origin/main, origin/sprint\_1\_staging, and HEAD. Then identify the merge type and briefly explain why it applies.

### Final graph

### Merge type and explanation

## 2.2 Explain the Repository State 10 points

The following graph shows Developer 1's local ExamScore repository. Read the graph carefully and answer each question.

```
A---B [main, origin/main]
  \
   C---D---E [feature/load_scores, origin/feature/load_scores]
     \
      F---G [HEAD -> sprint_1_staging]

origin/sprint_1_staging -> F
```

a. Which local branch is currently checked out, and which commit does HEAD identify? 2 points

Response

b. Which commit on sprint\_1\_staging exists only in this local repository? Explain how the graph shows this. 2 points

Response

c. Which commit supplies the files currently shown in the working directory? 1 point

Response

d. If `git merge feature/load_scores` is run now, can Git fast-forward sprint\_1\_staging? Explain the merge Git would need. 2 points

Response

**e. Does the graph prove that this merge will create a file conflict? Explain. 1 point**

**Response**

**f. What can git fetch origin update, and what will it not move automatically? 2 points**

**Response**

## 2.3 Write Commands and Redraw the Graph 10 points

Developer 1 begins with the following local repository state. Complete parts a through c in sequence. Each part begins from the result of the previous part.

```
A---B [main, origin/main]
  \
   C [sprint_1_staging, origin/sprint_1_staging]
    \
     D---E [HEAD -> feature/load_scores]
```

### Part a Switch to the Sprint Branch 2 points

Write the Git command to switch from feature/load\_scores to sprint\_1\_staging. Then redraw the graph to show the new HEAD position.

**Command**

**Graph after part a**

### Part b Merge the Feature Branch 3 points

Write the Git command to merge feature/load\_scores into the current sprint\_1\_staging branch. State the merge type and redraw the graph after the merge.

**Command and merge type**

**Graph after part b**

### Part c Stage and Commit Three File Changes 5 points

Using an editor, Developer 1 modifies `score_data.py`, modifies `scores.csv`, and creates `validation_notes.txt`. Write the Git commands to inspect the changes, stage only these three files, and create commit F with the message Document score validation cases. Then redraw the graph.

#### Commands

#### Graph after part c

### Problem 2.4 Predict a Fast Forward and New Commit 10 points

#### Category 1 Predict the graph after commands

Developer 1 begins with the following local ExamScore repository state.

```
A---B [main, origin/main]
  \
   C---D [sprint_1_staging, origin/sprint_1_staging]
     \
      E [HEAD -> feature/report_format]
```

#### Commands and Action

```
> git switch sprint_1_staging
> git merge feature/report_format
# Developer 1 edits report.py
> git add report.py
> git commit -m "Format summary report"
```

**Task** Draw the final repository graph. Show all local branches, remote-tracking references, and HEAD. Identify the merge type and explain which pointers move or remain unchanged.

## Problem 2.5 Predict a Three Way Merge and Final Switch 10 points

### Category 1 Predict the graph after commands

The feature and sprint branches have diverged after commit D. Assume the merge completes without a file conflict and creates merge commit G.

```
A---B [main, origin/main]
  \
   C---D---E [feature/basic_statistics, origin/feature/basic_statistics]
     \
      F [HEAD -> sprint_1_staging, origin/sprint_1_staging]
```

### Commands

```
> git merge feature/basic_statistics
> git switch main
```

**Task** Draw the final graph, including both parents of G. Show all branch pointers, remote-tracking references, and HEAD. Identify the merge type and explain why the final switch does not move any branch pointer.

## Problem 2.6 Explain a Local Main Branch Behind GitHub 10 points

### Category 2 Explain the graph

The graph shows one local repository after information from GitHub has already been fetched.

```
A---B---H [origin/main, v1.0]
  \
   C---D [sprint_1_staging, origin/sprint_1_staging]
     \
      E [HEAD -> feature/report_format]

main -> B
```

- Which branch is currently checked out, and which commit does HEAD identify? **2 points**
- Compare main with origin/main. What does the graph show about the local main branch? **2 points**
- If `git switch main` is run, will main automatically move to H? Explain where HEAD and main will be immediately afterward. **2 points**
- After switching to main, what command safely fast-forwards main to the known GitHub state at H? **2 points**
- Will updating main move `sprint_1_staging` or `feature/report_format`? **1 point**
- What does tag `v1.0` identify, and does switching branches move the tag? **1 point**

## Problem 2.7 Explain an Unpublished Merge Commit 10 points

### Category 2 Explain the graph

Developer 1 has integrated two feature histories into the local sprint branch.

```

A---B [main, origin/main]
  \
   C---D---E-----H [HEAD -> sprint_1_staging]
         \         /
          F---G-----/

feature/load_scores, origin/feature/load_scores -> E
feature/basic_statistics, origin/feature/basic_statistics -> G
origin/sprint_1_staging -> E

```

- Which branch is currently checked out, and which commit does HEAD identify? **1 point**
- Why is H a merge commit? Name its two parent commits. **2 points**
- Does sprint\_1\_staging at H contain the histories ending at both E and G? Explain. **2 points**
- Why does origin/sprint\_1\_staging still point to E instead of H? **2 points**
- Would git push origin feature/basic\_statistics publish H to sprint\_1\_staging on GitHub? Explain. **1 point**
- What command publishes H as the new GitHub sprint\_1\_staging branch state? **2 points**

## Problem 2.8 Create a Second Feature After Integration 10 points

### Category 3 Write commands and redraw after each step

Developer 1 starts with the following local Sprint 2 repository state. Complete the parts in sequence.

```

A---B [main, origin/main]
  \
   C [sprint_2_staging, origin/sprint_2_staging]
   \
    D---E [HEAD -> feature/pass_rate]

```

- Write the command to switch from feature/pass\_rate to sprint\_2\_staging. Redraw the graph with the revised HEAD position. **2 points**
- Merge feature/pass\_rate into sprint\_2\_staging. State the merge type and redraw the graph. **3 points**
- Create feature/grade\_distribution from the current commit using separate branch and switch commands. Redraw the graph. **2 points**
- Using an editor, modify report.py, modify scores.csv, and create grade\_ranges.csv. Stage only these files and create commit F with the message Add grade distribution report. Redraw the graph. **3 points**

## Problem 2.9 Update Staging Then Integrate a Diverged Feature 10 points

### Category 3 Write commands and redraw after each step

Developer 1 has a local sprint branch at D, a feature branch at F, and a newer known GitHub staging state at G. Complete the parts in sequence. Assume the final merge has no file conflict and creates commit H.

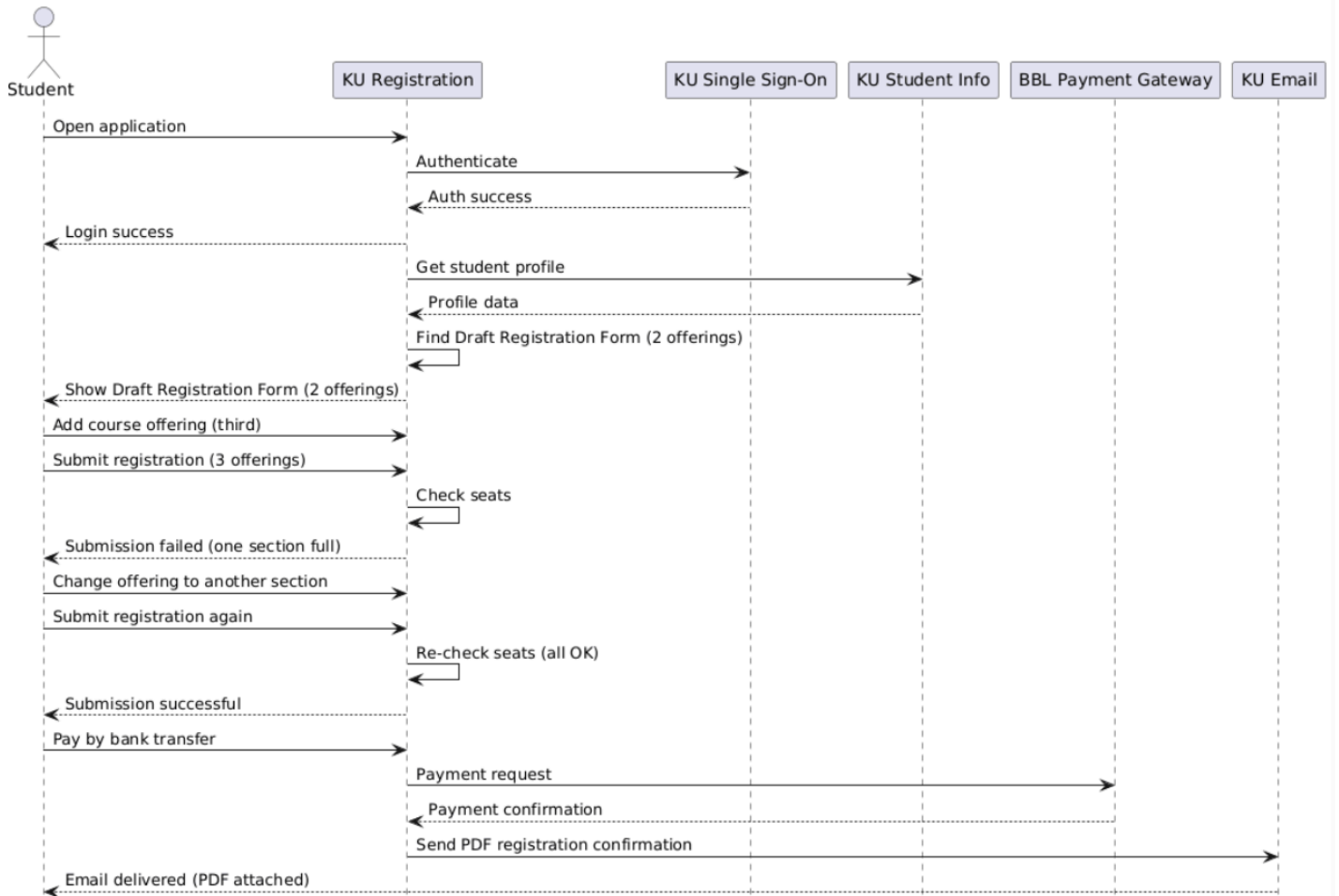
```
A---B [main, origin/main]
  \
   C---D---E---F [HEAD -> feature/export_results]
     \
      G [origin/sprint_2_staging]

sprint_2_staging -> D
```

- Switch to the local sprint\_2\_staging branch and redraw the graph with the revised HEAD position. **2 points**
- Update the current local staging branch to origin/sprint\_2\_staging using a fast-forward-only merge. Redraw the graph. **2 points**
- Merge feature/export\_results into the current staging branch. State the merge type and redraw the graph, including both parents of H. **4 points**
- Publish the integrated sprint\_2\_staging branch to GitHub and redraw the relevant local and remote-tracking pointers. **2 points**

# Solutions

**Solution 1.1:**



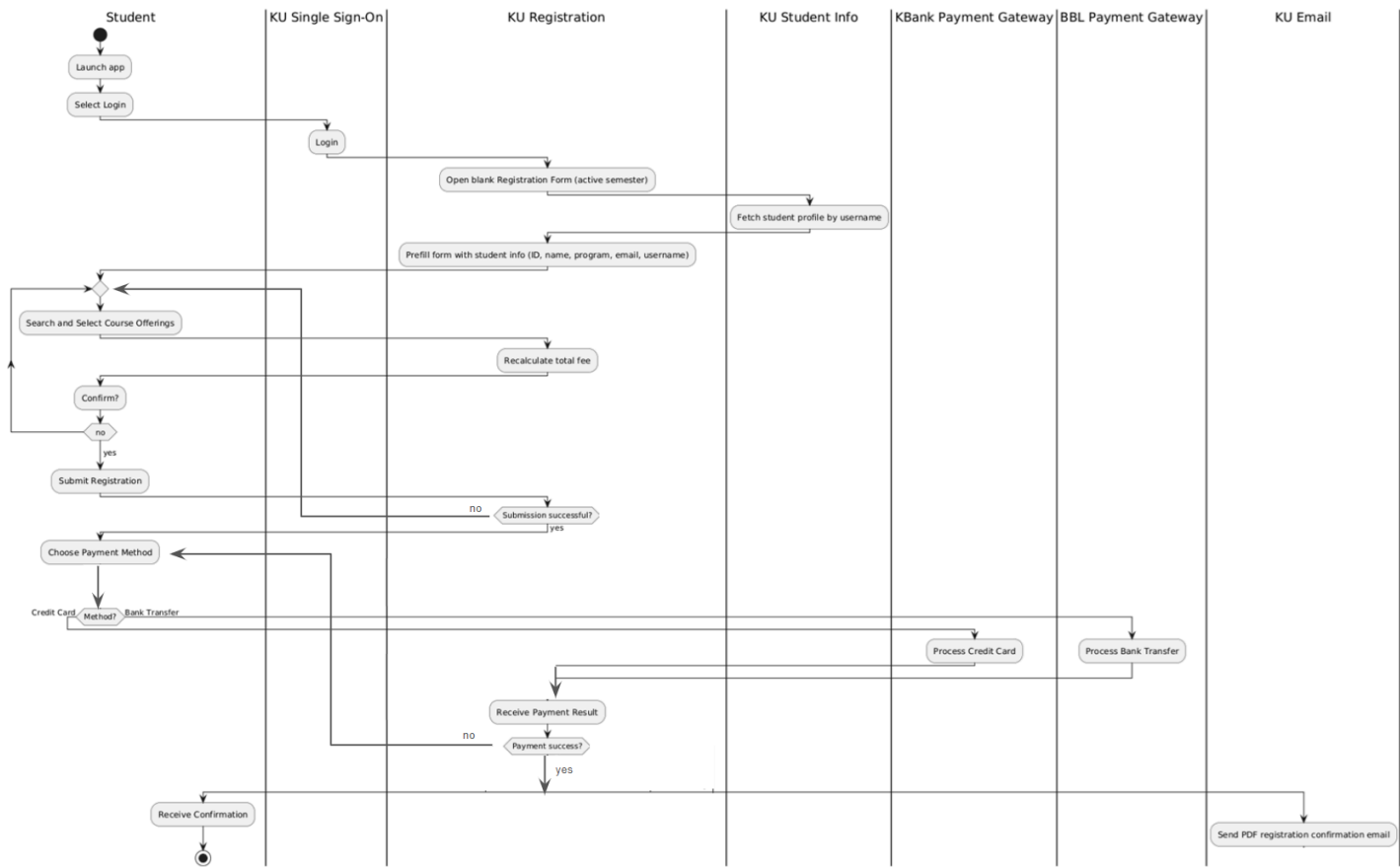
**Solution 1.2:**

| Student Registration Form View       |                                                       | User: Ying Kesuka | Login Name: Yingke                                   | Role: Student |
|--------------------------------------|-------------------------------------------------------|-------------------|------------------------------------------------------|---------------|
| [Close][Save][Clear] (Form Controls) |                                                       |                   |                                                      |               |
| Registration No.:                    | <input type="text" value="S2025-1-00232"/>            | *Date:            | <input type="text" value="_15/09/2025_"/> [Date]     |               |
| Username:                            | <input type="text" value="Yingke (from Login)"/>      |                   |                                                      |               |
| Student ID:                          | <input type="text" value="(Logged-in Student Info)"/> | Student Name:     | <input type="text" value="Ying Kesuka"/>             |               |
| Registration Semester:               | <input type="text" value="(from Active Semester)"/>   | Total Credits:    | <input type="text" value="15"/>                      |               |
| Total Price:                         | <input type="text" value="14,000.00"/>                | Current Status:   | <input type="text" value="Draft"/> [SUBMIT]          |               |
| Paid on Date:                        | <input type="text" value=""/> (autoset at “Done”)     | Paid By:          | <input type="text" value=""/> (autoset after SUBMIT) |               |

Enter Score and Grades for Students in this Course/Section:

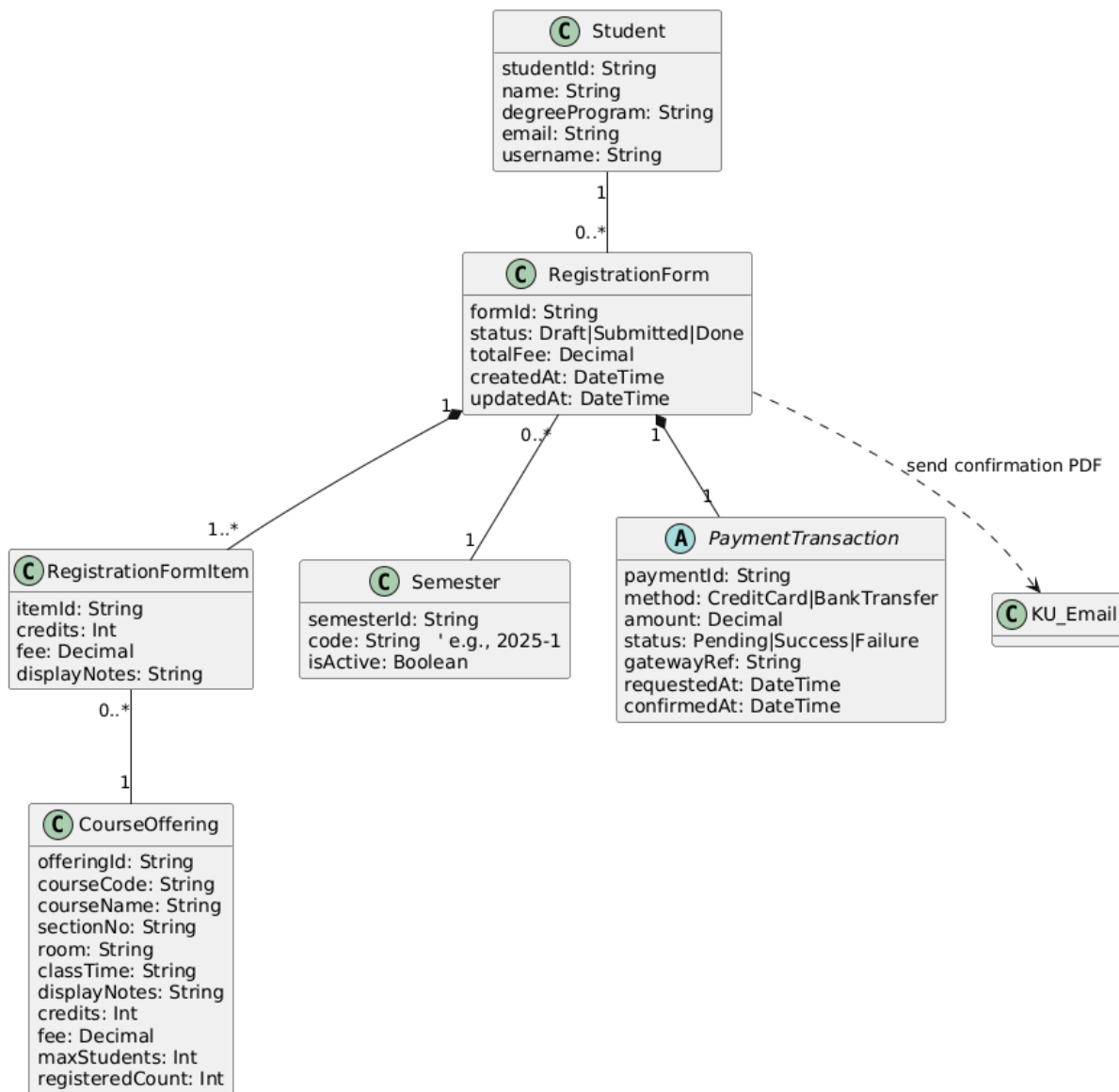
| # | *Course Offering ID | Course Code | Course Name          | Semester | Section | Credits | Max Students | Price    | Seats Remaining (view) |
|---|---------------------|-------------|----------------------|----------|---------|---------|--------------|----------|------------------------|
| 1 | 113 [LoV]           | CPE334      | Software Engineering | 2025-1   | 1       | 3       | 40           | 8,000.00 | 3                      |
| 2 | 119 [LoV]           | MTH233      | Calculus II          | 2025-1   | 7       | 4       | 40           | 6,000.00 | 12                     |
| 3 | [LoV]               |             |                      |          |         |         |              |          |                        |

Solution 1.3:



Solution 1.4:

### Registration Form Data Model — 1st-Level Links Only



## 2.1 Predict the New Repository Graph 10 points

### Starting Graph

```
A---B [main, origin/main]
  \
   C [sprint_1_staging, origin/sprint_1_staging]
    \
     D---E [HEAD -> feature/load_scores]
```

### Commands

```
> git switch sprint_1_staging
> git merge feature/load_scores
> git switch main
```

### Model Answer

```
A---B [HEAD -> main, origin/main]
  \
   C---D---E [sprint_1_staging, feature/load_scores]

origin/sprint_1_staging -> C
```

The first switch places HEAD on sprint\_1\_staging at C. Because C is an ancestor of E, the merge is a **fast-forward merge**. Git moves sprint\_1\_staging from C to E without creating a merge commit. feature/load\_scores remains at E. The final switch moves HEAD to main at B. The remote-tracking references do not move because none of the commands communicates with GitHub.

## 2.2 Explain the Repository State 10 points

```
A---B [main, origin/main]
  \
   C---D---E [feature/load_scores, origin/feature/load_scores]
     \
      F---G [HEAD -> sprint_1_staging]
```

origin/sprint\_1\_staging -> F

| Part     | Answer                                                                                                                                                                                           |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>a</b> | The current branch is sprint_1_staging, and HEAD identifies commit G.                                                                                                                            |
| <b>b</b> | Commit G exists only locally. The local sprint_1_staging pointer is at G, while origin/sprint_1_staging is still at F.                                                                           |
| <b>c</b> | Commit G supplies the files in the working directory because HEAD points through sprint_1_staging to G.                                                                                          |
| <b>d</b> | Git cannot fast-forward. feature/load_scores and sprint_1_staging diverged after their common ancestor C, so Git must perform a three-way merge and create a merge commit if the merge succeeds. |
| <b>e</b> | No. A commit graph shows history and ancestry, not which lines were changed. A conflict depends on incompatible file changes.                                                                    |
| <b>f</b> | git fetch origin may download new commits and update origin/* remote-tracking references. It does not move local branches, HEAD, or the working directory automatically.                         |

## 2.3 Write Commands and Redraw the Graph 10 points

### Starting Graph

```
A---B [main, origin/main]
  \
   C [sprint_1_staging, origin/sprint_1_staging]
    \
     D---E [HEAD -> feature/load_scores]
```

### Part a Model Answer

```
> git switch sprint_1_staging
```

```
A---B [main, origin/main]
  \
   C [HEAD -> sprint_1_staging, origin/sprint_1_staging]
    \
     D---E [feature/load_scores]
```

Only HEAD moves. No branch pointer moves merely because the developer switches branches.

### Part b Model Answer

```
> git merge feature/load_scores
```

```
A---B [main, origin/main]
  \
   C---D---E [HEAD -> sprint_1_staging, feature/load_scores]

origin/sprint_1_staging -> C
```

This is a fast-forward merge because sprint\_1\_staging at C is an ancestor of feature/load\_scores at E. The sprint branch moves to E, the feature branch remains at E, and no merge commit is created.

### Part c Model Answer

```
> git status
> git add score_data.py scores.csv validation_notes.txt
> git commit -m "Document score validation cases"
```

```
A---B [main, origin/main]
  \
   C---D---E---F [HEAD -> sprint_1_staging]
```

```
feature/load_scores -> E
origin/sprint_1_staging -> C
```

The file edits and file creation occur in the editor. `git add` places the three named file changes in the staging area. `git commit` creates F and advances the current branch and HEAD to F. `feature/load_scores` remains at E, and `origin/sprint_1_staging` remains at C because no push or fetch occurred.

#### Solution 2.4 Predict a Fast Forward and New Commit

##### Final Graph

```
A---B [main, origin/main]
  \
   C---D---E---F [HEAD -> sprint_1_staging]

feature/report_format -> E
origin/sprint_1_staging -> D
```

Switching places HEAD on `sprint_1_staging` at D. Because D is an ancestor of E, the merge fast-forwards `sprint_1_staging` to E without creating a merge commit. Commit F is then created on the current sprint branch. The feature pointer remains at E, and `origin/sprint_1_staging` remains at D because no remote command is run.

#### Solution 2.5 Predict a Three Way Merge and Final Switch

##### Final Graph

```
A---B [HEAD -> main, origin/main]
  \
   C---D---F---G [sprint_1_staging]
       \   /
        E-----/

feature/basic_statistics, origin/feature/basic_statistics -> E
origin/sprint_1_staging -> F
```

E and F both descend from D, so neither branch tip is an ancestor of the other. Git performs a three-way merge and creates G with parents F and E. `sprint_1_staging` moves to G. The final switch places HEAD on main at B but does not move main or any other branch pointer.

#### Solution 2.6 Explain a Local Main Branch Behind GitHub

| Part | Model answer                                                              |
|------|---------------------------------------------------------------------------|
| a    | <code>feature/report_format</code> is checked out, and HEAD identifies E. |

| Part     | Model answer                                                                                                                     |
|----------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>b</b> | Local main is at B while origin/main is at H. Because H is a child of B, local main is one commit behind the known GitHub state. |
| <b>c</b> | No. git switch main places HEAD on main at B. Switching does not move main to H.                                                 |
| <b>d</b> | Run git merge --ff-only origin/main while main is checked out. main and HEAD then move to H.                                     |
| <b>e</b> | No. sprint_1_staging remains at D and feature/report_format remains at E.                                                        |
| <b>f</b> | v1.0 is a fixed tag naming commit H. Switching branches does not move the tag.                                                   |

### Solution 2.7 Explain an Unpublished Merge Commit

| Part     | Model answer                                                                                                               |
|----------|----------------------------------------------------------------------------------------------------------------------------|
| <b>a</b> | sprint_1_staging is checked out, and HEAD identifies H.                                                                    |
| <b>b</b> | H has two parents, E and G, so H is a merge commit.                                                                        |
| <b>c</b> | Yes. E and G are both ancestors of H, so the sprint branch contains both histories.                                        |
| <b>d</b> | origin/sprint_1_staging records the last known GitHub staging state. H exists only locally because it has not been pushed. |
| <b>e</b> | No. That command pushes feature/basic_statistics, whose tip is G. It does not update GitHub's sprint_1_staging branch.     |
| <b>f</b> | Run git push origin sprint_1_staging.                                                                                      |

## Solution 2.8 Create a Second Feature After Integration

### Part a

```
> git switch sprint_2_staging
```

```
A---B [main, origin/main]
  \
   C [HEAD -> sprint_2_staging, origin/sprint_2_staging]
    \
     D---E [feature/pass_rate]
```

### Part b

```
> git merge feature/pass_rate
```

```
A---B [main, origin/main]
  \
   C---D---E [HEAD -> sprint_2_staging, feature/pass_rate]

origin/sprint_2_staging -> C
```

The merge is a fast-forward because C is an ancestor of E.

### Part c

```
> git branch feature/grade_distribution
> git switch feature/grade_distribution
```

```
A---B [main, origin/main]
  \
   C---D---E [sprint_2_staging, feature/pass_rate,
              HEAD -> feature/grade_distribution]

origin/sprint_2_staging -> C
```

## Part d

```
> git add report.py scores.csv grade_ranges.csv  
> git commit -m "Add grade distribution report"
```

```
A---B [main, origin/main]  
  \  
    C---D---E---F [HEAD -> feature/grade_distribution]  
  
sprint_2_staging, feature/pass_rate -> E  
origin/sprint_2_staging -> C
```

## Solution 2.9 Update Staging Then Integrate a Diverged Feature

### Part a

```
> git switch sprint_2_staging
```

```
A---B [main, origin/main]  
  \  
    C---D---E---F [feature/export_results]  
      \  
        G [origin/sprint_2_staging]  
  
HEAD -> sprint_2_staging -> D
```

### Part b

```
> git merge --ff-only origin/sprint_2_staging
```

```
A---B [main, origin/main]  
  \  
    C---D---E---F [feature/export_results]  
      \  
        G [HEAD -> sprint_2_staging, origin/sprint_2_staging]
```

### Part c

```
> git merge feature/export_results
```

```
A---B [main, origin/main]
  \
   C---D---G---H [HEAD -> sprint_2_staging]
        \   /
         E---F
```

```
feature/export_results -> F
origin/sprint_2_staging -> G
```

The histories ending at F and G diverged after D. Git therefore performs a three-way merge and creates H with parents G and F.

### Part d

```
> git push origin sprint_2_staging
```

```
A---B [main, origin/main]
  \
   C---D---G---H [HEAD -> sprint_2_staging,
                  origin/sprint_2_staging]
        \   /
         E---F
```

```
feature/export_results -> F
```